

 AVALIAÇÃO

ACADÊMICO(A):		VALOR:	30,0 pts
MATRÍCULA:		DATA:	___/___/2026
DISCIPLINA:	Estrutura de Dados	CURSO / PER.:	Ciência da Computação • 2º Período
PROFESSOR:	Prof. Me. Karan	ETAPA:	N1 DISCURSIVA

FOLHA DE RESPOSTAS — ENTREGA OBRIGATÓRIA (LINHAS PAUTADAS)

- Responda às questões exclusivamente nas linhas pautadas desta folha (Frente e Verso), identificando com clareza o número da questão e o subitem (ex: **QUESTÃO 01 - a**).
- Utilize caneta azul ou preta. Apenas esta primeira folha deve ser entregue. O Caderno de Questões anexo fica com o acadêmico.

FOLHA DE RESPOSTAS (FRENTE) — LINHAS 01 A 25

01	
02	
03	
04	
05	
06	
07	
08	
09	
10	
11	
12	
13	
14	
15	
16	
17	
18	
19	
20	
21	
22	
23	
24	
25	

FOLHA DE RESPOSTAS (VERSO) — LINHAS 26 A 65

26	
27	
28	
29	
30	
31	
32	
33	
34	
35	
36	
37	
38	
39	
40	
41	
42	
43	
44	
45	
46	
47	
48	
49	
50	
51	
52	
53	
54	
55	
56	
57	
58	
59	
60	
61	
62	
63	
64	
65	

QUESTÃO 01 (RASTREAMENTO DE MEMÓRIA)*Apontamento Múltiplo Cruzado, Aritmética e Modificação Indireta (7,5 pts)*

O programa abaixo em linguagem C executa uma sequência de transferências e cálculos na memória RAM manipulando um vetor estático através de múltiplos ponteiros que alternam seus alvos:

```
#include <stdio.h>

int main() {
    int vet[4] = {5, 15, 25, 35};
    int *p1 = &vet[0];
    int *p2 = &vet[2];
    int *p3 = p1;

    *p3 = *p2 - *p1;
    p1 = p1 + 1;
    *p1 = (*p3) * 2;
    p3 = p2;
    *p2 = *(p1 - 1) + *(p3);
    p2 = &vet[3];
    *p2 = *p1 + *p3;

    printf("vet: [%d, %d, %d, %d] ", vet[0], vet[1], vet[2], vet[3]);
    return 0;
}
```

a) (4,0 pts) Demonstre na Folha de Respostas o **Teste de Mesa completo** (desenhando a tabela com as gavetas de memória e setas de apontamento), acompanhando instrução por instrução para onde `p1`, `p2` e `p3` apontam e quais são os valores atribuídos a cada posição do vetor `vet`. Ao final, apresente a **saída exata impressa pelo printf** no console.

b) (3,5 pts) Explique detalhadamente na Folha de Respostas:

1. Qual é a diferença fundamental entre a instrução `p3 = p2;` e a instrução `*p3 = *p2;` em termos de registradores e células de memória RAM?
2. O que teria acontecido em nível de hardware se, na linha `p1 = p1 + 1;`, o programador tivesse digitado por engano `p1 = p1 + 5;` e em seguida executado `*p1 = 10;`? Explique o conceito de *violação de segmento / ponteiro solto (wild pointer)*.

QUESTÃO 02 (PASSAGEM POR REFERÊNCIA CRUZADA)*Escopo de Funções, Parâmetros Cruzados e Isolamento na Stack (7,5 pts)*

O programa abaixo executa uma função que combina simultaneamente passagem por valor e passagem por referência, realizando inclusive chamadas cruzadas com a mesma variável em posições de parâmetros distintas:

```
#include <stdio.h>

void calcular(int *a, int b, int *c, int *d) {
    int temp = *a + b;
    *a = *c * 2;
    *c = temp - b;
    b = b * 3;
    *d = *a + *c + b;
}

int main() {
    int x = 4, y = 6, z = 8, w = 2;

    // Chamada 1:
    calcular(&x, y, &z, &w);
    printf("C1: x=%d, y=%d, z=%d, w=%d ", x, y, z, w);

    // Chamada 2 (Cruzada):
    calcular(&x, x, &y, &z);
    printf("C2: x=%d, y=%d, z=%d, w=%d ", x, y, z, w);

    return 0;
}
```

a) (4,0 pts) Rastreie o Teste de Mesa linha por linha para a **Chamada 1** e para a **Chamada 2**. Apresente as duas linhas exatas de saída impressas no console (`C1` e `C2`).

b) (3,5 pts) Na Chamada 2, a variável `x` foi passada simultaneamente no primeiro parâmetro (`&x`) e no segundo parâmetro (`x`). Explique detalhadamente como a pilha de execução (Stack) tratou essa variável: por que a alteração de `*a = *c * 2;` dentro da função não modificou o valor de `b` durante a mesma execução?

QUESTÃO 03 (LÓGICA PROPOSICIONAL DE 3 VARIÁVEIS) *Transitividade da Condicional, Tabela Verdade de 8 Linhas e Tautologia (7,5 pts)*

Em controle de fluxo e árvores de decisão, a propriedade transitiva da implicação lógica (conhecida como Silogismo Hipotético) garante que regras encadeadas de negócios preservem consistência dedutiva. Considere a proposição composta contendo 3 variáveis proposicionais independentes (P, Q, R):

$$[(P \rightarrow Q) \wedge (Q \rightarrow R)] \rightarrow (P \rightarrow R)$$

a) (4,50 pts) Construa e resolva na Folha de Respostas a **tabela verdade completa com as 8 combinações** ($2^3 = 8$ linhas: VVV, VVF, VFV, VFF, FVV, FVF, FFV, FFF), demonstrando passo a passo a resolução de cada coluna intermediária:

$P \rightarrow Q$ • $Q \rightarrow R$ • $(P \rightarrow Q) \wedge (Q \rightarrow R)$ • $P \rightarrow R$ • **Coluna Final**.

b) (3,00 pts) Com base na coluna final, classifique formalmente essa proposição como **Tautologia**, **Contradição** ou **Contingência**, justificando com rigor o conceito. Em seguida, indique como os conectivos *Conjunção* ($\wedge$), *Disjunção* ($\vee$) e *Negação* ($\neg$) são representados na linguagem C e explique por que a implicação $P \rightarrow Q$ é logicamente equivalente à expressão $\neg P \vee Q$.

QUESTÃO 04 (CONTROLE DE VERSÃO E RESOLUÇÃO DE CONFLITO)

Ciclo de Vida do Git, Inspeção de Status e Merge Conflict em C (7,5 pts)

Dois desenvolvedores estão colaborando no desenvolvimento de um módulo em C. Durante o trabalho, um deles executou o comando de verificação e obteve a seguinte saída no terminal:

```
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   operacoes.c
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    teste.c
```

a) (3,5 pts) Com base **exclusivamente na saída do terminal acima**:

- Identifique em qual das três áreas locais do Git (*Working Directory*, *Staging Area* ou *Local Repository*) encontra-se atualmente o arquivo `operacoes.c` e em qual área encontra-se o arquivo `teste.c`.
- Explique a diferença conceitual entre o estado *Changes to be committed (Staged)* e o estado *Untracked files*. Em seguida, informe os comandos exatos de terminal necessários para: (I) consolidar `operacoes.c` no histórico local com a mensagem "Ajusta operacoes"; e (II) passar a rastrear o arquivo `teste.c` na área de preparação.

Em seguida, ao tentar sincronizar as alterações com o repositório remoto via `git pull`, o Git detectou alterações concorrentes e gerou um conflito no arquivo `operacoes.c`, gravando os marcadores abaixo:

```
int processar(int valor) {
<<<<<< HEAD
    // Alteração local na máquina:
    return valor * 2 + 10;
=====
    // Alteração recebida do GitHub:
    return (valor < 0) ? 0 : valor * 3;
>>>>>> 8f3c4e2a1b9d
}
```

b) (4,0 pts) Com base no trecho de conflito impresso acima:

- Explique o significado técnico de cada um dos três marcadores gerados pelo Git (`<<<<<< HEAD`, `=====` e `>>>>>> 8f3c4e2a1b9d`).
- Suponha que a equipe decidiu unificar a lógica aplicando a multiplicação por 3 mais a soma de 10 (`valor * 3 + 10`), preservando a regra de que valores negativos devem retornar 0. Apresente na Folha de Respostas o **código C final da função processar** (totalmente limpo, sem marcadores) e indique a **seqüência exata de comandos no terminal** para concluir a mesclagem e enviar a versão final ao GitHub.